AutoRestTest: An AI-Powered Platform for Automated REST API Testing

Dependency-aware multi-agent reinforcement learning + LLMs, delivered as a collaborative web platform

SOFTWARE PROJECT LAB 3

Mohammad Ismail Hossain (BSSE 1433)
Institute of Information Technology (IIT)
University of Dhaka
Supervisor: **Toukir Ahammed**
Lecturer, IIT, University of Dhaka

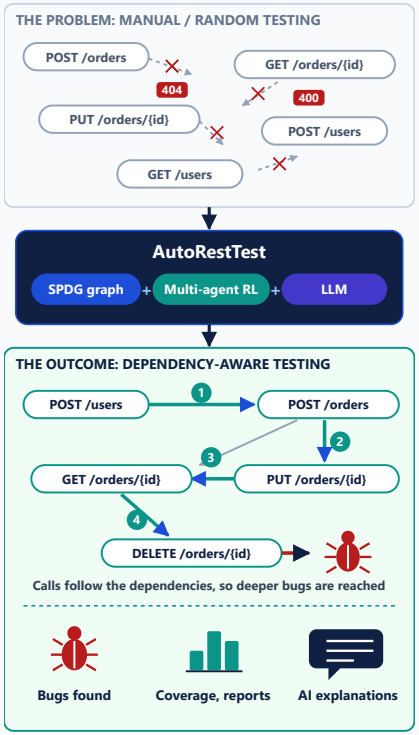
Scan for source code (GitHub)

1 Problem & Motivation

- **Huge input space:** endless parameter, payload and value combinations.
- **Operation dependencies:** the ID from `POST /orders` feeds `GET /orders/{id}`, so a test needs a *valid sequence*.
- **Weak status quo:** manual tests are slow and do not scale. Random fuzzing ignores dependencies and misses deep bugs.

Goal: automate REST API testing end to end, from spec to bug explanation.

2 Graphical Abstract



3 Methodology

STAGE 1: SPEC GENERATION FROM SOURCE CODE (OOPS + LLM agents; skipped if a spec is uploaded)

1 Source .zip — project upload → 2 Tech analysis — language, framework → 3 Entry detection — scan files for APIs → 4 File deps — cross-file refs → 5 Endpoints — HTTP method + path → 6 Schemas — request / response

Assemble OpenAPI 3.x document → staged for review → USER approves and applies (never silent)

STAGE 2: INITIALIZATION (outside the time budget)

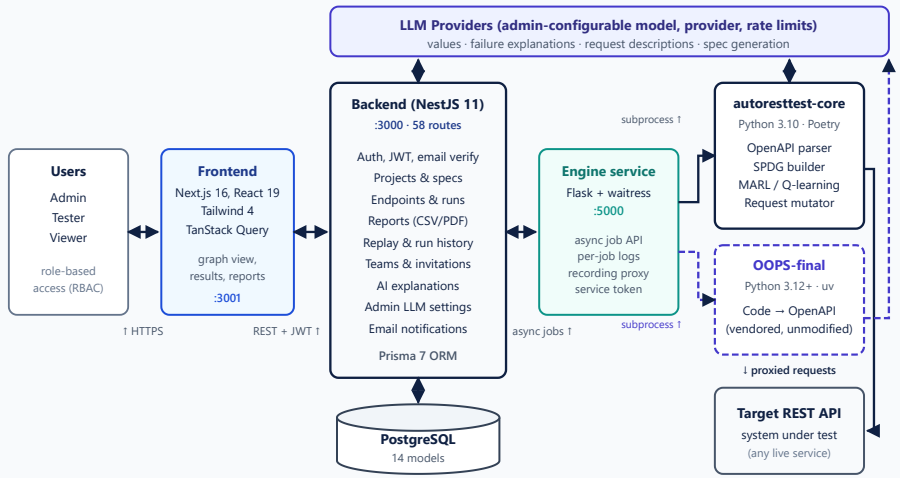
OpenAPI 3.x spec — uploaded or generated → Parse and validate — operations, schemas → Build SPDG — cosine-similarity edges → Init Q-tables — + LLM-made values

STAGE 3: TESTING LOOP (until the time budget is spent)

Agents choose — Operation · Parameter · Value · Body · Data-source · Dependency (+ opt-in Header agent) → Request generator — uses SPDG sources → Mutator — rate 0.2: bad types, boundaries, no auth → Target REST API — response / status code — **5xx → bug recorded**

reward: update Q-tables and refine SPDG

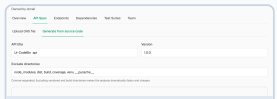
4 System Architecture (High-Level)



5 Key Features

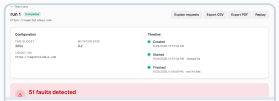
- **Spec from code** — Upload OpenAPI 3.x or generate it from zipped code; you review it first.
- **Dependency graph** — Shows which operations feed others, with learned confidence.
- **AI-driven testing** — Q-learning agents learn valid call sequences; ~20% of requests are mutated.
- **Request capture** — Stores every request and response; copy as curl or re-send.
- **AI explanations** — Plain-language reason for each failed request.
- **Replay** — Re-run exact requests to confirm a fix.
- **Reports** — Coverage and status-code results; CSV/PDF export.
- **Teams and roles** — Admin, Tester and Viewer roles, enforced on the server.

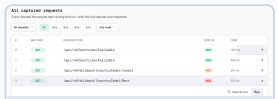
6 Platform Screenshots

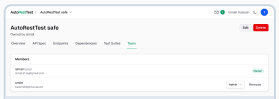

Spec generation from code


Dependency graph


Test run setup


Test results


Captured request log


Team management

7 Research Basis

[1] A Multi-Agent Approach for REST API Testing with Semantic Graphs and LLM-Driven Inputs, *IEEE/ACM ICSE 2025* (arXiv:2411.07098) — Testing engine foundation.
[2] OOPS: Automated generation of REST API specification via LLMs (arXiv:2601.12735) — Specification generation pipeline.